

Step 1:

Prompt 1:

I have to integrate JIRA with User story-tests generator and fetch the userstories by mentioning spaces Name

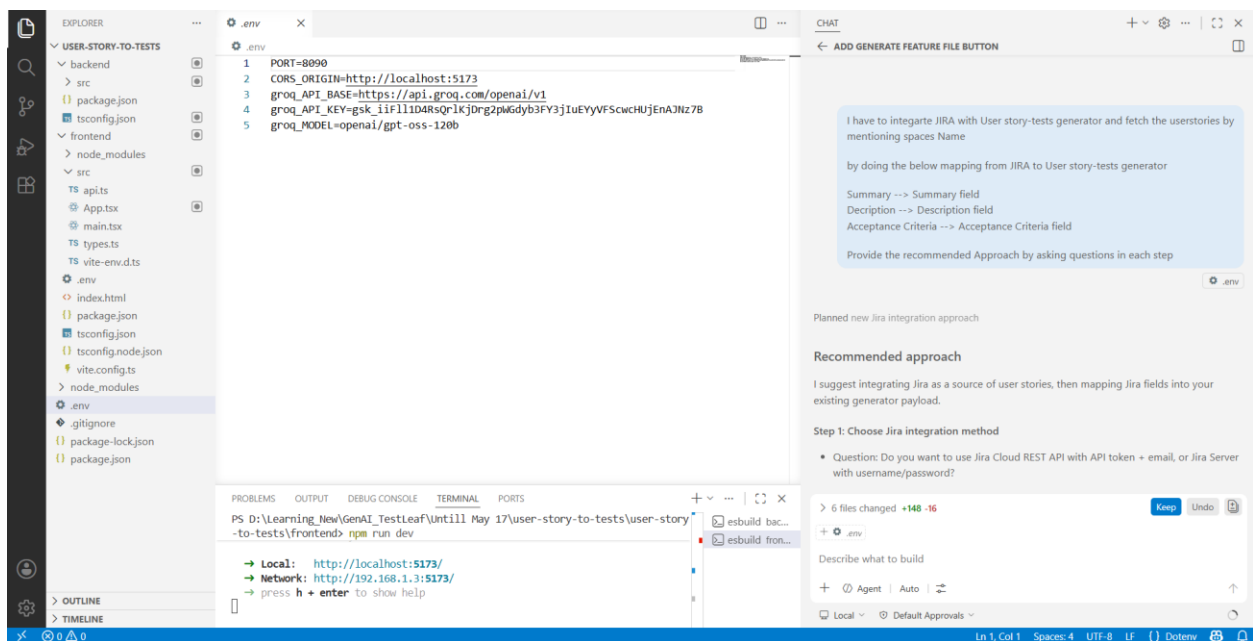
by doing the below mapping from JIRA to User story-tests generator

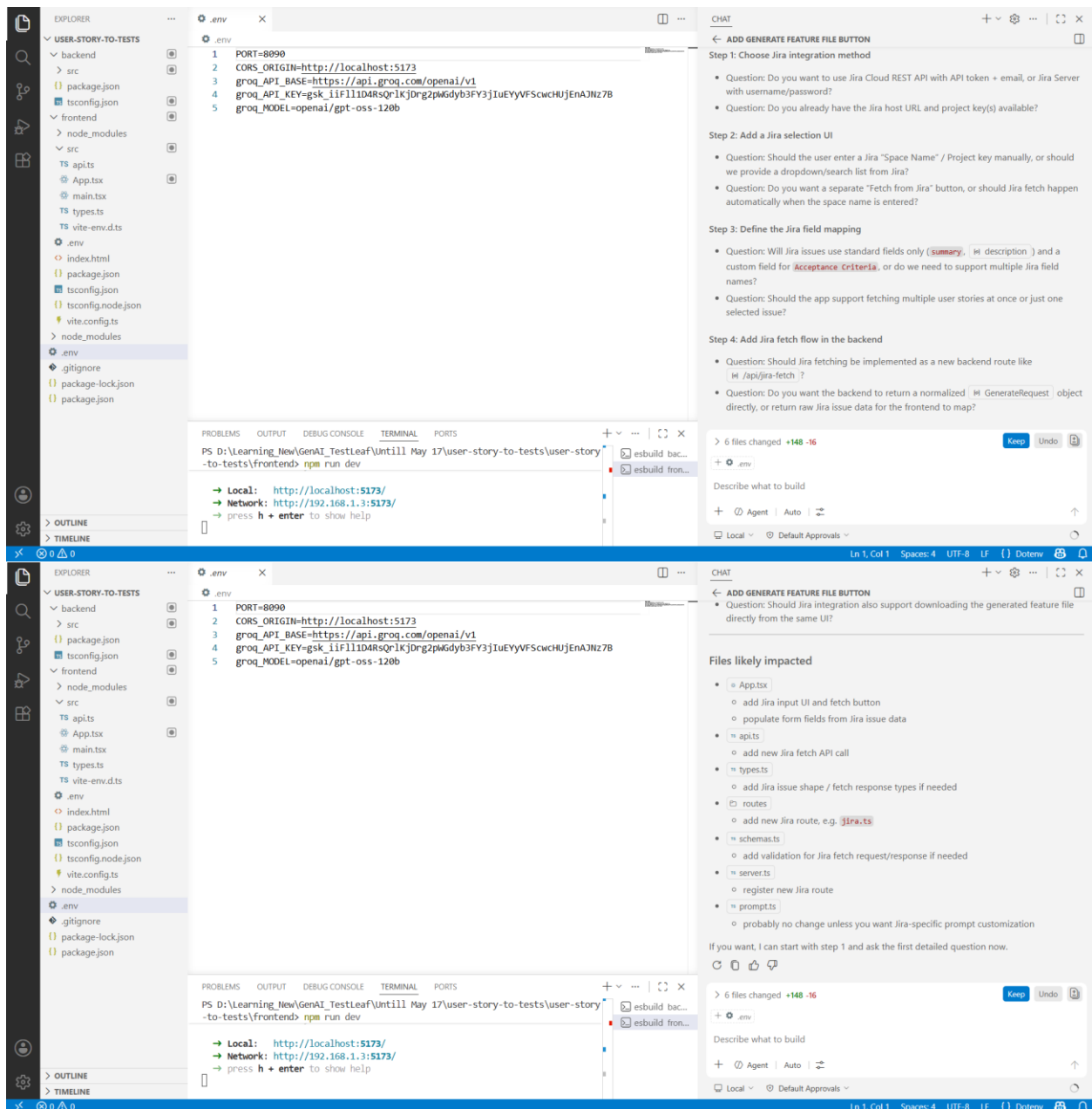
Summary --> Summary field

Description --> Description field

Acceptance Criteria --> Acceptance Criteria field

Provide the recommended Approach by asking questions in each step





Step 2:

Prompt 2:

Implement the recommended approach both in the Frontend and Backend in each of the step ask questions one by one.

EXPLORER

- USER-STORY-TO-TESTS
 - backend
 - src
 - package.json
 - tsconfig.json
 - frontend
 - node_modules
 - src
 - TS apits
 - App.tsx
 - main.tsx
 - types.ts
 - vite-env.d.ts
 - .env
 - index.html
 - package.json
 - tsconfig.json
 - tsconfig.node.json
 - vite.config.ts
 - node_modules
 - .env
 - gitignore
 - package-lock.json
 - package.json

TS apits

```
5 export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {
19   const data: GenerateResponse = await response.json()
20   return data
21 } catch (error) {
22   console.error('Error generating tests:', error)
23   throw error instanceof Error ? error : new Error('Unknown error occurred')
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Learning_New\GenAI_TestLeaf\Until May 17\user-story-to-tests\user-story> npm run dev

CHAT

ADD GENERATE FEATURE FILE BUTTON

Refining the Jira project selection behavior so no issue fetch happens when the dropdown is cleared.

Updated App.tsx and considered improvements +8 -2

Jira integration implemented

 - Added Jira Server integration to both frontend and backend.

What's now in place

 - Frontend:
 - Load Jira Projects button
 - Jira project dropdown populated from Jira
 - Jira story dropdown showing only issues with type **Story**
 - Fetch Jira Story button to populate:
 - Summary -> Story Title
 - Description
 - Acceptance Criteria
 - Form fields remain editable after Jira fetch
 - Backend:
 - New route file: `api.ts`
 - Jira API endpoints:
 - GET `/api/jira/projects`

9 files changed +468 -19

TS apits

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 29, Col 8 Spaces: 2 UTF-8 LF TypeScript

EXPLORER

- USER-STORY-TO-TESTS
 - backend
 - src
 - package.json
 - tsconfig.json
 - frontend
 - node_modules
 - src
 - TS apits
 - App.tsx
 - main.tsx
 - types.ts
 - vite-env.d.ts
 - .env
 - index.html
 - package.json
 - tsconfig.json
 - tsconfig.node.json
 - vite.config.ts
 - node_modules
 - .env
 - gitignore
 - package-lock.json
 - package.json

TS apits

```
5 export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {
19   const data: GenerateResponse = await response.json()
20   return data
21 } catch (error) {
22   console.error('Error generating tests:', error)
23   throw error instanceof Error ? error : new Error('Unknown error occurred')
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\Learning_New\GenAI_TestLeaf\Until May 17\user-story-to-tests\user-story> npm run dev

CHAT

ADD GENERATE FEATURE FILE BUTTON

Jira story dropdown showing only issues with type **Story**

 - Fetch Jira Story button to populate:
 - Summary -> Story Title
 - Description
 - Acceptance Criteria
 - Form fields remain editable after Jira fetch

Backend:

 - New route file: `api.ts`
 - Jira API endpoints:
 - GET `/api/jira/projects`
 - GET `/api/jira/issues?projectId={projectId}`
 - GET `/api/jira/issue/{issueKey}={issueKey}`
 - Parses Jira `description` text for headings:
 - Summary
 - Description
 - Acceptance Criteria

server.ts updated to mount `/api/jira`

Files impacted

 - App.tsx
 - api.ts
 - types.ts

9 files changed +468 -19

TS apits

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 29, Col 8 Spaces: 2 UTF-8 LF TypeScript

EXPLORER

USER-STORY-TO-TESTS

backend

src

package.json

tsconfig.json

frontend

node_modules

src

apits

App.tsx

main.tsx

types.ts

vite-env.d.ts

env

index.html

package.json

tsconfig.json

tsconfig.node.json

vite.config.ts

node_modules

env

gitignore

package-lock.json

package.json

TS apits

generateFeature

export async function generateTests(request: GenerateRequest): Promise<GenerateResponse> {

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

const data: GenerateResponse = await response.json()

return data

catch (error) {

console.error('Error generating tests:', error)

throw error instanceof Error ? error : new Error('unknown error occurred')

}

export async function generateFeature(request: GenerateRequest): Promise<string> {

try {

const response = await fetch(`\${API\_BASE\_URL}/generate-feature`, {

method: 'POST',

headers: {

'Content-Type': 'application/json',

},

body: JSON.stringify(request),

})

if (!response.ok) {

const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))

throw new Error(errorData.error || 'HTTP error! status: \${response.status}')

}

const featureText = await response.text()

return featureText

catch (error) {

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS D:\Learning_New\GenAI_TestLeaf\Until May 17\user-story-to-tests\user-story-to-tests\frontend> npm run dev

esbuild bac...

esbuild fron...

CHAT

ADD GENERATE FEATURE FILE BUTTON

backend:

New route file: = jira.ts

Jira API endpoints:

GET /api/jira/projects

GET /api/jira/issues?projectId={projectId}

GET /api/jira/issue?issueKey={issueKey}

Parses Jira

description

text for headings:

Summary:

Description:

Acceptance Criteria:

server.ts updated to mount /api/jira

Files impacted

App.tsx

apits

types.ts

jira.ts

server.ts

Status

No diagnostics errors were reported in the updated files.

9 files changed +468 -19

TS apits

Describe what to build

Agent

Auto

Local

Default Approvals

Ln 29, Col 8 Spaces: 2 UTF-8 LF TypeScript

EXPLORER

USER-STORY-TO-TESTS

backend

src

package.json

tsconfig.json

frontend

node_modules

src

apits

App.tsx

main.tsx

types.ts

vite-env.d.ts

env

index.html

package.json

tsconfig.json

tsconfig.node.json

vite.config.ts

node_modules

env

gitignore

package-lock.json

package.json

TS apits

generateFeature

export async function fetchJiraProjects(): Promise<JiraProject[]> {

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

const response = await fetch(`\${API\_BASE\_URL}/jira/projects`)

if (!response.ok) {

const errorData = await response.json().catch(() => ({ error: 'Unknown error' }))

throw new Error(errorData.error || 'HTTP error! status: \${response.status}')

}

const data = await response.json()

return data.projects as JiraProject[]

}

export async function fetchJiraIssues(projectKey: string): Promise<JiraIssue[]> {

const response = await fetch(`\${API\_BASE\_URL}/jira/issue/\${projectKey}`)

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS D:\Learning_New\GenAI_TestLeaf\Until May 17\user-story-to-tests\user-story-to-tests\frontend> npm run dev

node backen...

esbuild fron...

CHAT

ADD GENERATE FEATURE FILE BUTTON

please proceed

Considered implementation steps and inspected App.tsx

Done

I proceeded with the Jira project loading fix.

What changed

apits

updated default backend URL fallback to http://localhost:8080/api

App.tsx

added JiraProjectsLoaded state

improved UI logic so the Jira project dropdown only appears after a successful load

added a helper message when no projects are found after load

Result

Load Jira Projects now correctly fetches Jira projects

the dropdown will appear when projects are returned

compile diagnostics are clean in the updated files

9 files changed +477 -20

TS apits

Describe what to build

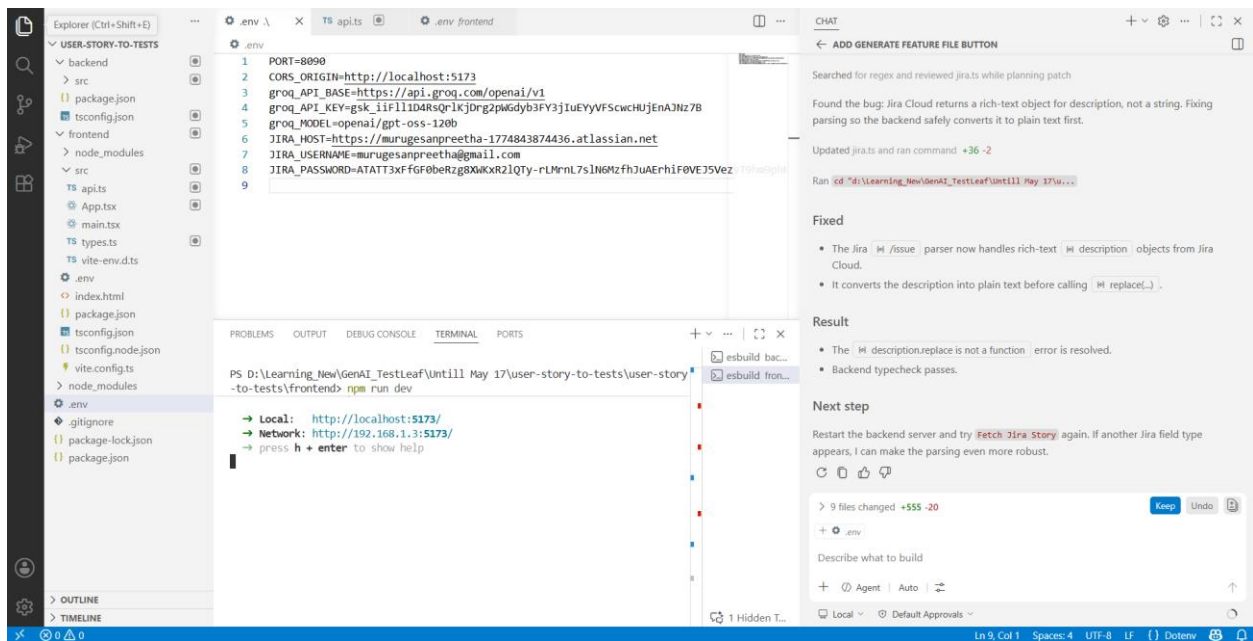
Agent

Auto

Local

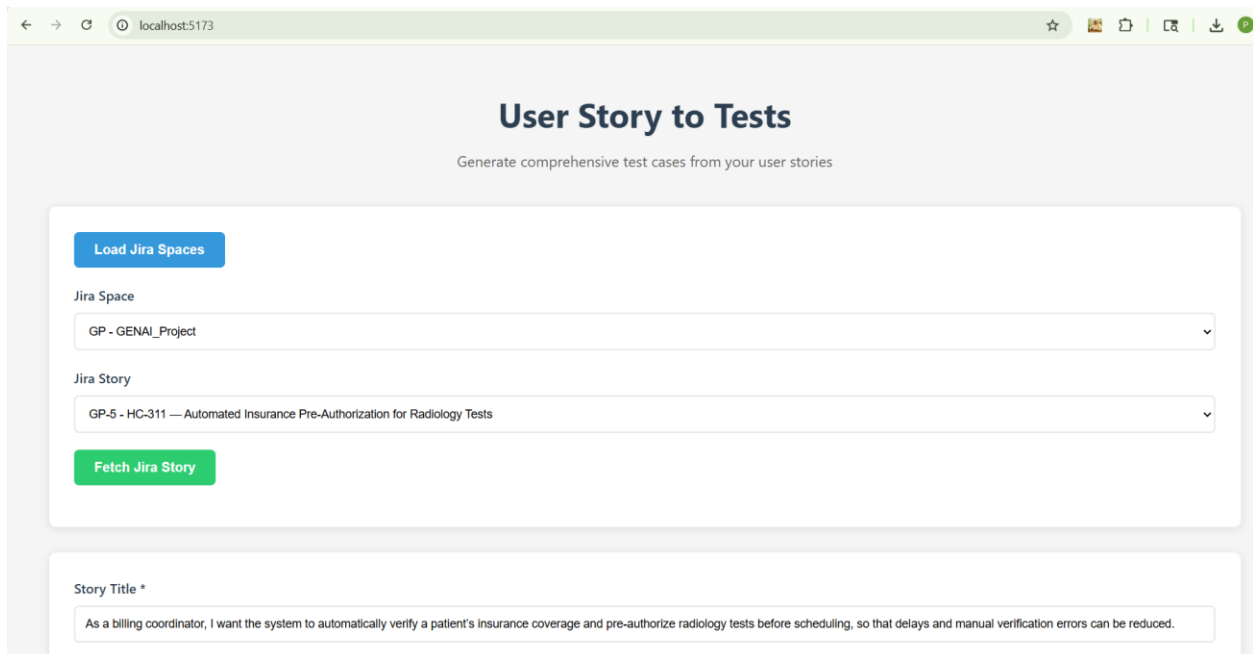
Default Approvals

Ln 29, Col 8 Spaces: 2 UTF-8 LF TypeScript



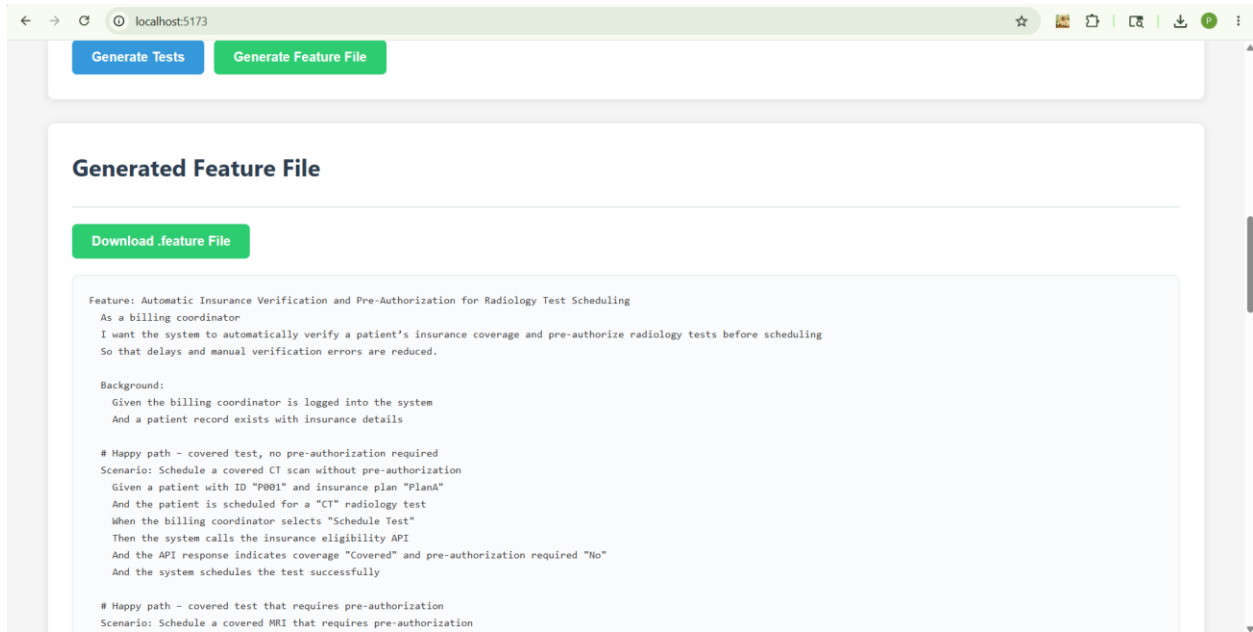
Step 3:

Output:



The screenshot shows a web application interface. At the top, there's a browser address bar with 'localhost:5173'. Below it, there are two buttons: 'Generate Tests' (blue) and 'Generate Feature File' (green). The main content area is titled 'Generated Test Cases' in a large, bold font. Below the title, a subtitle reads '12 test case(s) generated • Model: openai/gpt-oss-120b • Tokens: 2311'. A table follows, displaying the generated test cases. The table has four columns: 'Test Case ID', 'Title', 'Category', and 'Expected Result'. There are four rows of test cases, each with a blue arrow icon to the left of the ID. The first three rows have a 'Positive' category, and the last row has a 'Negative' category.

Test Case ID	Title	Category	Expected Result
▶ TC-001	Verify insurance eligibility for a covered radiology test	Positive	System calls insurance API, receives eligibility confirmation, and allows scheduling to proceed
▶ TC-002	Successful pre-authorization for a test that requires it	Positive	System sends pre-authorization request, receives approval ID and date, stores them in the billing record, and confirms scheduling
▶ TC-003	Notify coordinator and patient when test is not covered	Positive	System receives a not-covered response, displays a notification to the coordinator, sends an email/SMS to the patient with alternative payment options
▶ TC-004	Handle insurance API timeout gracefully	Negative	System shows a timeout error message, logs the incident, and provides options to retry or cancel scheduling



Downloaded Feature file content:

```
Feature: Automatic Insurance Verification and Pre-Authorization for Radiology Test Scheduling
  As a billing coordinator
  I want the system to automatically verify a patient's insurance coverage and pre-authorize radiology tests
  before scheduling
  So that delays and manual verification errors are reduced.

  Background:
    Given the billing coordinator is logged into the system
    And a patient record exists with insurance details
  # Happy path - covered test, no pre-authorization required
  Scenario: Schedule a covered CT scan without pre-authorization
    Given a patient with ID "P001" and insurance plan "PlanA"
    And the patient is scheduled for a "CT" radiology test
    When the billing coordinator selects "Schedule Test"
    Then the system calls the insurance eligibility API
    And the API response indicates coverage "Covered" and pre-authorization required "No"
    And the system schedules the test successfully
  # Happy path - covered test that requires pre-authorization
  Scenario: Schedule a covered MRI that requires pre-authorization
    Given a patient with ID "P002" and insurance plan "PlanB"
    And the patient is scheduled for an "MRI" radiology test
    When the billing coordinator selects "Schedule Test"
    Then the system calls the insurance eligibility API
    And the API response indicates coverage "Covered" and pre-authorization required "Yes"
    And the system initiates the pre-authorization process
    And the system stores the authorization ID "AUTH98765" and approval date "2024-05-10" in the billing record
    And the coordinator receives a confirmation of successful pre-authorization
  # Error flow - test not covered
  Scenario: Attempt to schedule an uncovered Ultrasound
    Given a patient with ID "P003" and insurance plan "PlanC"
    And the patient is scheduled for an "Ultrasound" radiology test
    When the billing coordinator selects "Schedule Test"
    Then the system calls the insurance eligibility API
    And the API response indicates coverage "Not Covered" and pre-authorization required "No"
    Then the system notifies the coordinator and the patient that the test is not covered
    And the system offers alternative payment options
  # Error flow - insurance API failure
  Scenario: Insurance eligibility API failure handling
    Given a patient with ID "P004" and insurance plan "PlanD"
    And the patient is scheduled for a "CT" radiology test
    When the billing coordinator selects "Schedule Test"
    Then the system attempts to call the insurance eligibility API
    And the API returns an error "Service Unavailable"
    Then the system displays an error message to the coordinator and logs the incident
  # Error flow - invalid procedure code
  Scenario: Invalid radiology procedure code handling
    Given a patient with ID "P005" and insurance plan "PlanE"
```

And the billing coordinator enters an invalid procedure code "XYZ123"
When the billing coordinator selects "Schedule Test"
Then the system validates the procedure code
And the system returns an error "Procedure code XYZ123 is not recognized"

Data-driven eligibility verification
Scenario Outline: Insurance eligibility verification for various radiology procedures
Given a patient with ID "<PatientId>" and insurance plan "<InsurancePlan>"
And the patient is scheduled for a "<Procedure>" radiology test
When the billing coordinator selects "Schedule Test"
Then the system calls the insurance eligibility API
And the API response indicates coverage "<CoverageStatus>" and pre-authorization required "<AuthRequired>"
And the system processes the response resulting in "<ResultAction>"
Examples:

PatientId	InsurancePlan	Procedure	CoverageStatus	AuthRequired	ResultAction
P006	PlanF	CT	Covered	No	Test scheduled successfully
P007	PlanG	MRI	Covered	Yes	Pre-authorization initiated
P008	PlanH	Ultrasound	Not Covered	No	Coordinator and patient notified, alternative payment offered
P009	PlanI	CT	Error	No	Error displayed to coordinator
P010	PlanJ	MRI	Covered	Yes	Pre-authorization failed, notification sent

Data-driven pre-authorization handling
Scenario Outline: Pre-authorization handling and billing record update
Given a patient with ID "<PatientId>" and insurance plan "<InsurancePlan>"
And the patient is scheduled for a "<Procedure>" radiology test that requires pre-authorization
When the system sends a pre-authorization request to the insurer
Then the insurer returns a response with status "<AuthStatus>" and ID "<AuthId>"
And the system stores the authorization ID "<StoredAuthId>" and approval date "<ApprovalDate>" in the billing record
And the system notifies the coordinator with "<NotificationMessage>"
Examples:

PatientId	InsurancePlan	Procedure	AuthStatus	AuthId	StoredAuthId	ApprovalDate	NotificationMessage
P011	PlanK	MRI	Approved	AUTH123	AUTH123	2024-05-01	Pre-authorization approved and recorded
P012	PlanL	CT	Denied	AUTH456			Pre-authorization denied, alternative options suggested
P013	PlanM	Ultrasound	Pending	AUTH789			Pre-authorization pending, coordinator notified to follow up
P014	PlanN	MRI	Error				Pre-authorization request failed, error logged

Data-driven insurance data validation
Scenario Outline: Handling missing or invalid insurance information
Given a patient with ID "<PatientId>" and insurance information "<InsuranceInfo>"
And the patient is scheduled for a "<Procedure>" radiology test
When the billing coordinator selects "Schedule Test"
Then the system validates insurance data
And the system responds with "<ErrorMessage>"
Examples:

PatientId	InsuranceInfo	Procedure	ErrorMessage
P015	None	CT	Insurance information missing
P016	Expired	MRI	Insurance policy expired
P017	UnknownProvider	Ultrasound	Insurance provider not recognized
P018	InvalidPlanCode	CT	Insurance plan code invalid

Edge case - API timeout
Scenario: Insurance eligibility API timeout handling
Given a patient with ID "P019" and insurance plan "PlanTimeout"
And the patient is scheduled for a "CT" radiology test
When the billing coordinator selects "Schedule Test"
Then the system attempts to call the insurance eligibility API
And the API does not respond within the timeout period
Then the system displays a timeout error to the coordinator and logs the incident

Edge case - boundary dates for coverage
Scenario Outline: Coverage verification on policy boundary dates
Given a patient with ID "<PatientId>" and insurance plan "<InsurancePlan>" whose coverage end date is "<CoverageEndDate>"
And the patient is scheduled for a "<Procedure>" radiology test on "<ScheduledDate>"
When the billing coordinator selects "Schedule Test"
Then the system calls the insurance eligibility API
And the API response indicates coverage "<CoverageStatus>"
And the system processes the result as "<ResultAction>"
Examples:

PatientId	InsurancePlan	CoverageEndDate	ScheduledDate	Procedure	CoverageStatus	ResultAction
P020	PlanO	2024-05-31	2024-05-30	CT	Covered	Test scheduled successfully

P021	PlanP	2024-05-31	2024-06-01	MRI	Not Covered	Coordinator
and patient notified, payment offered						
P022	PlanQ	2024-02-29	2024-02-28	Ultrasound	Covered	Test
scheduled successfully						

Backend information:

PS D:\Learning_New\GenAI_TestLeaf\Untill May 17\user-story-to-tests\user-story-to-tests\backend> npm run dev

```
> backend@1.0.0 dev
> tsx watch src/server.ts
```

Loading .env from: D:\Learning_New\GenAI_TestLeaf\Untill May 17\user-story-to-tests\user-story-to-tests\.env

Environment variables loaded:

PORT: 8090

CORS_ORIGIN: http://localhost:5173

groq_API_BASE: https://api.groq.com/openai/v1

groq_API_KEY: SET

groq_MODEL: openai/gpt-oss-120b

Backend server running on port 8090

API available at http://localhost:8090/api

Health check at http://localhost:8090/api/health

Groq API key configured successfully

Using model: openai/gpt-oss-120b

API endpoint: https://api.groq.com/openai/v1/chat/completions

Making Groq API call:

Endpoint: https://api.groq.com/openai/v1/chat/completions

Model: openai/gpt-oss-120b

API Key: gsk_iIF1l1...Nz7B

Request body: {

"model": "openai/gpt-oss-120b",

"messages": [

{

"role": "system",

"content": "You are a senior QA engineer with expertise in creating comprehensive test cases from user stories. Your task is to analyze user stories and generate detailed test cases.\n\nCRITICAL: You must return ONLY valid JSON matching this exact schema:\n{\n \"cases\": [\n {\n \"id\": \"TC-001\", \n \"title\": \"string\", \n \"steps\": [\"string\", \"...\"], \n \"testData\": \"string (optional)\", \n \"expectedResult\": \"string\", \n \"category\": \"string (e.g., Positive|Negative|Edge|Authorization|Non-Functional)\", \n \"model\": \"string (optional)\", \n \"promptTokens\": 0, \n \"completionTokens\": 0\n }\n], \n \"guidelines\": \"string (optional)\", \n \"imperativeSteps\": [\"Click login button\", \"Enter valid email\"]\n}\n\nGuidelines:\n- Generate test case IDs like TC-001, TC-002, etc.\n- Write concise, imperative steps (e.g., \"Click login button\", \"Enter valid email\")\n- Include Positive, Negative, and Edge test cases where relevant\n- Categories: Positive, Negative, Edge, Authorization, Non-Functional\n- Steps should be actionable and specific\n- Expected results should be clear and measurable\n\nReturn ONLY the JSON object, no additional text or formatting."

},

{

"role": "user",

"content": "Generate comprehensive test cases for the following user story:\n\nStory Title: As a billing coordinator, I want the system to automatically verify a patient's insurance coverage and pre-authorize radiology tests before scheduling, so that delays and manual verification errors can be reduced.\n\nAcceptance Criteria:\nGiven a patient is scheduled for a radiology test, When the billing coordinator selects "Schedule Test", Then the system should verify insurance eligibility via API.\nGiven the test requires pre-authorization, When the request is sent, Then the system should store the authorization ID and approval date in the billing record.\nGiven a test is not covered, When the result is received, Then the system should notify both the coordinator and patient, and offer alternative payment options.\n\nDescription:\nBefore scheduling any CT, MRI, or ultrasound procedure, the system should trigger an API call to the insurance provider to check if the procedure is covered under the patient's plan. If authorization is required, it should initiate the pre-authorization process automatically and store the approval ID in the patient record.\n\nEpic: Financial & Billing Workflows - Insurance Automation\n\nGenerate test cases covering positive scenarios, negative scenarios, edge cases, and any authorization or non-functional requirements as applicable. Return only the JSON response."

},

"temperature": 0.2

}

Response status: 200 OK

Response headers: {

'alt-svc': 'h3=":443"; ma=86400',

'cache-control': 'private, max-age=0, no-store, no-cache, must-revalidate',

'cf-cache-status': 'DYNAMIC',

'cf-ray': '9fe2bb46cbdf8826-MAA',

connection: 'keep-alive',

'content-encoding': 'br',

'content-type': 'application/json',

date: 'Tue, 19 May 2026 11:23:51 GMT',

server: 'cloudflare',

```
'set-cookie': '__cf_bm=Dqp.xsHO07M.YotD23d9mpWfR0c47deCNgXYa5Dlhzc-1779189827.6440916-1.0.1.1-EfVi154Y3en.icTrsJ4zrREG.1WUW4xrDk25321FDBeVID0Wz0c3YgvgNhd91WEdbSEWBmW4Jm3QYaf4Utq0f1Q20Wyu98Wak5QTHHau04Tl.CwzDe3GJH_.DpIV5UaR; HttpOnly; SameSite=None; Secure; Path=/; Domain=groq.com; Expires=Tue, 19 May 2026 11:53:51 GMT',
'strict-transport-security': 'max-age=15552000',
'transfer-encoding': 'chunked',
'vary': 'Origin',
'via': '1.1 google',
'x-groq-region': 'bom',
'x-ratelimit-limit-requests': '1000',
'x-ratelimit-limit-tokens': '8000',
'x-ratelimit-remaining-requests': '999',
'x-ratelimit-remaining-tokens': '7437',
'x-ratelimit-reset-requests': '1m26.4s',
'x-ratelimit-reset-tokens': '4.222s',
'x-request-id': 'req_01krzzmj39emqvfnd631zmnsvv'
}
```

```
Success response: {
  "id": "chatcmpl-3eae382-8eec-4049-b5b1-9ede7b0ed6d0",
  "object": "chat.completion",
  "created": 1779189831,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "{\n  \"cases\": [\n    {\n      \"id\": \"TC-001\", \"title\": \"Schedule covered radiology test with successful insurance eligibility verification\", \"steps\": [\n        \"Log in as billing coordinator\", \"Navigate to patient record\", \"Select a CT scan that is covered by the patient's insurance plan\", \"Click 'Schedule Test' button\", \"testData\": \"Patient ID: 12345, Procedure: CT Head, Insurance Plan: XYZ Gold\", \"expectedResult\": \"System calls insurance eligibility API, receives a positive coverage response, and displays confirmation that the test is eligible\", \"category\": \"Positive\", \"id\": \"TC-002\", \"title\": \"Schedule radiology test requiring pre-authorization and store authorization details\", \"steps\": [\n          \"Log in as billing coordinator\", \"Open patient record for a procedure that needs pre-authorization (e.g., MRI with contrast)\", \"Click 'Schedule Test'\", \"System sends pre-authorization request to insurance API\", \"Verify that the response contains an authorization ID and approval date\", \"Confirm that the authorization ID and approval date are saved in the billing record\", \"testData\": \"Patient ID: 67890, Procedure: MRI Spine with Contrast, Insurance Plan: ABC Premium\", \"expectedResult\": \"Authorization ID and approval date are stored in the patient's billing record and visible to the coordinator\", \"category\": \"Positive\", \"id\": \"TC-003\", \"title\": \"Attempt to schedule a test that is not covered by insurance\", \"steps\": [\n          \"Log in as billing coordinator\", \"Select a patient with a plan that does not cover ultrasound\", \"Click 'Schedule Test' for an ultrasound procedure\", \"System receives a negative coverage response from the insurance API\", \"testData\": \"Patient ID: 11223, Procedure: Abdominal Ultrasound, Insurance Plan: Basic Plan\", \"expectedResult\": \"System notifies the coordinator and the patient that the test is not covered and displays alternative payment options\", \"category\": \"Negative\", \"id\": \"TC-004\", \"title\": \"Handle insurance API timeout during eligibility check\", \"steps\": [\n          \"Log in as billing coordinator\", \"Select any radiology test\", \"Click 'Schedule Test'\", \"Simulate insurance API not responding within timeout period\", \"testData\": \"API response time > 30 seconds\", \"expectedResult\": \"System shows an error message indicating verification could not be completed and allows the coordinator to retry or cancel scheduling\", \"category\": \"Negative\", \"id\": \"TC-005\", \"title\": \"Verify system behavior when insurance API returns malformed JSON\", \"steps\": [\n          \"Log in as billing coordinator\", \"Select a radiology test\", \"Click 'Schedule Test'\", \"Mock insurance API to return invalid JSON payload\", \"testData\": \"API response time > 30 seconds\", \"expectedResult\": \"System logs the parsing error, displays a user-friendly error message, and prevents test scheduling until a valid response is received\", \"category\": \"Edge\", \"id\": \"TC-006\", \"title\": \"Schedule test when patient has multiple active insurance policies\", \"steps\": [\n          \"Log in as billing coordinator\", \"Open patient record with primary and secondary insurance\", \"Select a radiology test\", \"Click 'Schedule Test'\", \"System queries primary insurance first; if not covered, queries secondary insurance\", \"testData\": \"Patient ID: 33445, Primary Plan: Not Covered, Secondary Plan: Covered\", \"expectedResult\": \"System successfully verifies coverage using secondary insurance and proceeds with scheduling\", \"category\": \"Positive\", \"id\": \"TC-007\", \"title\": \"Attempt scheduling without required coordinator permissions\", \"steps\": [\n          \"Log in as a user with read-only access\", \"Navigate to patient record\", \"Attempt to click 'Schedule Test'\", \"testData\": \"API response time > 30 seconds\", \"expectedResult\": \"System prevents the action and displays an authorization error message\", \"category\": \"Authorization\", \"id\": \"TC-008\", \"title\": \"Verify audit log entry is created after successful pre-authorization\", \"steps\": [\n          \"Log in as billing coordinator\", \"Schedule a test that requires pre-authorization\", \"Complete the pre-authorization process\", \"Access system audit log for the patient\", \"testData\": \"API response time > 30 seconds\", \"expectedResult\": \"Audit log contains an entry with timestamp, coordinator ID, procedure, authorization ID, and approval date\", \"category\": \"Non-Functional\", \"id\": \"TC-009\", \"title\": \"Performance test: eligibility verification completes within acceptable time\", \"steps\": [\n          \"Log in as billing coordinator\", \"Select a radiology test\", \"Click 'Schedule Test'\", \"Measure time from click to receipt of eligibility response\", \"testData\": \"API response time > 30 seconds\", \"expectedResult\": \"Eligibility verification completes in <= 2 seconds under normal load\", \"category\": \"Non-Functional\", \"id\": \"TC-010\", \"title\": \"Schedule test\"
        ]
      }
    }
  ]
}
```

```

when insurance provider returns a pending authorization status\",\\n      \\\"steps\\\": [\\n      \\\"Log in as
billing coordinator\\\",\\n      \\\"Select a procedure that triggers pre-authorization\\\",\\n      \\\"Click
\\\\\\\\\"Schedule Test\\\\\\\\\"\\\",\\n      \\\"Mock insurance API to return status = \\\\\\\\\\"Pending\\\\\\\\\" with a temporary
authorization ID\\\"\\n      ],\\n      \\\"expectedResult\\\": \\\"System records the pending status, notifies
coordinator that approval is pending, and does not finalize scheduling until final approval is received\\\",\\n
\\\"category\\\": \\\"Edge\\\"\\n      ],\\n      {\\n      \\\"id\\\": \\\"TC-011\\\",\\n      \\\"title\\\": \\\"System handles concurrent
scheduling requests for the same patient\\\",\\n      \\\"steps\\\": [\\n      \\\"Log in as two billing coordinators
on separate sessions\\\",\\n      \\\"Both select the same radiology test for the same patient\\\",\\n      \\\"Both
click \\\\\\\\\\"Schedule Test\\\\\\\\\" simultaneously\\\"\\n      ],\\n      \\\"expectedResult\\\": \\\"System processes one
request successfully and the second receives a conflict message indicating the test is already scheduled\\\",\\n
\\\"category\\\": \\\"Edge\\\"\\n      ],\\n      {\\n      \\\"id\\\": \\\"TC-012\\\",\\n      \\\"title\\\": \\\"Verify that patient
notification email contains correct alternative payment options when test is not covered\\\",\\n      \\\"steps\\\":
[\\n      \\\"Log in as billing coordinator\\\",\\n      \\\"Attempt to schedule an uncovered test\\\",\\n
\\\"System generates patient notification\\\",\\n      \\\"Check email content sent to patient\\\"\\n      ],\\n
\\\"expectedResult\\\": \\\"Email includes clear alternative payment options (e.g., self-pay rates, financing) and
reference to the test and patient ID\\\",\\n      \\\"category\\\": \\\"Positive\\\"\\n      }\\n      ],\\n      \\\"model\\\": \\\"gpt-4o-
mini-2024-07-18\\\",\\n      \\\"promptTokens\\\": 0,\\n      \\\"completionTokens\\\": 0\\n}\\\",
      \\\"reasoning\\\": \\\"We need to output JSON with \\\"cases\\\": [ ... ], \\\"model\\\": optional, \\\"promptTokens\\\": 0,
\\\"completionTokens\\\": 0. Must include test cases with IDs, title, steps array, testData optional,
expectedResult, category.\\n\\nWe need to cover positive, negative, edge, authorization, non-functional.\\n\\nLet's
craft maybe 10-12 test cases.\\n\\nTest case IDs sequential.\\n\\nWill include testData field where needed.\\n\\nWe
need to ensure JSON is valid, no extra text.\\n\\nLet's produce.\\n\\n\"
    },
    \\\"logprobs\\\": null,
    \\\"finish_reason\\\": \\\"stop\"
  }
},
\\\"usage\\\": {
  \\\"queue_time\\\": 0.055692471,
  \\\"prompt_tokens\\\": 562,
  \\\"prompt_time\\\": 0.028834268,
  \\\"completion_tokens\\\": 1602,
  \\\"completion_time\\\": 3.380292001,
  \\\"total_tokens\\\": 2164,
  \\\"total_time\\\": 3.409126269,
  \\\"completion_tokens_details\\\": {
    \\\"reasoning_tokens\\\": 106
  }
},
\\\"usage_breakdown\\\": null,
\\\"system_fingerprint\\\": \\\"fp_a09bde29de\\\",
\\\"x_groq\\\": {
  \\\"id\\\": \\\"req_01krzzmj39emqvfdn631zmsvsv\\\",
  \\\"seed\\\": 78651346
},
\\\"service_tier\\\": \\\"on_demand\"
}

```

```

Groq API key configured successfully
Using model: openai/gpt-oss-120b
API endpoint: https://api.groq.com/openai/v1/chat/completions
Groq API key configured successfully
Using model: openai/gpt-oss-120b
API endpoint: https://api.groq.com/openai/v1/chat/completions

```

🔑 Making Groq API call:

🔑 Endpoint: https://api.groq.com/openai/v1/chat/completions

🔑 Model: openai/gpt-oss-120b

🔑 API Key: gsk_iiFl11...Nz7B

```

📄 Request body: {
  \\\"model\\\": \\\"openai/gpt-oss-120b\\\",
  \\\"messages\\\": [
    {

```

```

      \\\"role\\\": \\\"system\\\",
      \\\"content\\\": \\\"You are a senior QA engineer with expertise in creating comprehensive test cases from user
stories. Your task is to analyze user stories and generate detailed test cases.\\n\\nCRITICAL: You must return
ONLY valid JSON matching this exact schema:\\n\\n{\\n  \\\"cases\\\": [\\n    {\\n      \\\"id\\\": \\\"TC-001\\\",\\n
\\\"title\\\": \\\"string\\\",\\n      \\\"steps\\\": [\\\"string\\\", \\\"...\\\"],\\n      \\\"testData\\\": \\\"string (optional)\\\",\\n
\\\"expectedResult\\\": \\\"string\\\",\\n      \\\"category\\\": \\\"string (e.g., Positive|Negative|Edge|Authorization|Non-
Functional)\\\"\\n    }\\n  ],\\n  \\\"model\\\": \\\"string (optional)\\\",\\n  \\\"promptTokens\\\": 0,\\n
\\\"completionTokens\\\": 0\\n}\\n\\nGuidelines:\\n- Generate test case IDs like TC-001, TC-002, etc.\\n- Write concise,
imperative steps (e.g., \\\"Click login button\\\", \\\"Enter valid email\\\")\\n- Include Positive, Negative, and Edge
test cases where relevant\\n- Categories: Positive, Negative, Edge, Authorization, Non-Functional\\n- Steps
should be actionable and specific\\n- Expected results should be clear and measurable\\n\\nReturn ONLY the JSON
object, no additional text or formatting.\"
    },
    {

```

```

      \\\"role\\\": \\\"user\\\",
      \\\"content\\\": \\\"Generate comprehensive test cases for the following user story:\\n\\nStory Title: As a billing
coordinator, I want the system to automatically verify a patient's insurance coverage and pre-authorize
radiology tests before scheduling, so that delays and manual verification errors can be reduced.\\n\\nAcceptance
Criteria:\\nGiven a patient is scheduled for a radiology test,When the billing coordinator selects \\\"Schedule

```

Test",Then the system should verify insurance eligibility via API.\nGiven the test requires pre-authorization,When the request is sent,Then the system should store the authorization ID and approval date in the billing record.\nGiven a test is not covered,When the result is received,Then the system should notify both the coordinator and patient, and offer alternative payment options.\n\nDescription:\n\nBefore scheduling any CT, MRI, or ultrasound procedure, the system should trigger an API call to the insurance provider to check if the procedure is covered under the patient's plan. If authorization is required, it should initiate the pre-authorization process automatically and store the approval ID in the patient record.\nEpic:Financial & Billing Workflows - Insurance Automation\n\nGenerate test cases covering positive scenarios, negative scenarios, edge cases, and any authorization or non-functional requirements as applicable. Return only the JSON response."

```

    },
    "temperature": 0.2
  }
}

```

Response status: 200 OK

Story Title *

As a billing coordinator, I want the system to automatically verify a patient's insurance coverage and pre-authorize rax

Description

Before scheduling any CT, MRI, or ultrasound procedure, the system should trigger an API call to the insurance provider to check if the procedure is covered under the patient's plan. If authorization is required, it should initiate the pre-authorization process automatically and store the approval ID in the patient record.
Epic:Financial & Billing Workflows - Insurance Automation

Acceptance Criteria *

Given a patient is scheduled for a radiology test,when the billing coordinator selects "Schedule Test",Then the system should verify insurance eligibility via API.
Given the test requires pre-authorization,when the request is sent,Then the system should store the authorization ID and approval date in the billing record.
Given a test is not covered,when the result is received,Then the system should notify both

Additional Info

Any additional information (optional)...

Generate Tests **Generate Feature File**

Network Log

2 / 3 requests

1 issue76s...
2 generate...

Request 1: generate...

Method: POST
URL: /api/insurance/verify
Status: 200 OK

Response:

```

{
  "id": "TC-004",
  "title": "Handle insurance API timeout (Negative)",
  "steps": [
    "Log in as billing coordinator",
    "Open patient record",
    "Select any radiology test",
    "Simulate insurance API timeout",
    "Click 'Schedule Test'"
  ],
  "expectedResult": "System shows a timeout error",
  "category": "Negative"
},
{
  "id": "TC-005",
  "title": "Reject scheduling when patient insurance is not verified",
  "steps": [
    "Log in as billing coordinator",
    "Open patient record missing policy number",
    "Select a CT scan",
    "Click 'Schedule Test'"
  ],
  "expectedResult": "System validates input, displays error message"
}

```

Request 2: generate...

Method: POST
URL: /api/insurance/verify
Status: 200 OK

Response:

```

{
  "id": "TC-006",
  "title": "Verify insurance eligibility via API",
  "steps": [
    "Log in as billing coordinator",
    "Open patient record",
    "Select any radiology test",
    "Click 'Schedule Test'"
  ],
  "expectedResult": "System returns 'Covered' status",
  "category": "Positive"
}

```

